UNIT 2 – NETWORK MODELS AND PROTOCOLS AND DATA COMMUNICATION

PRN:2125UCSM1118
MINI-PROJECT REPORT

Protocol Analysis with Wireshark

Student Name	Yash Kawade
Project	Protocol Analysis with Wireshark
Software	Wireshark (v4.x) & Mozilla Firefox
Environment	Kali Linux (eth0)
Target Server	neverssl.com (34.223.124.45:80)
GitHub Repository	wireshark
GitHub Link	https://github.com/kawdeyash/Wireshark.git

Submitted as part of Unit 2 practical / mini-project

1. Objective

To capture and analyze live network packets using Wireshark on Kali Linux, examine the TCP three-way handshake (SYN, SYN-ACK, ACK), and observe the subsequent HTTP GET request and HTTP 200 OK response.

2. Software and Requirements

Device / Software	Quantity	Purpose
Kali Linux	1	Operating system and network platform
Wireshark (v4.x)	1	Network packet analyzer and deep inspection
Mozilla Firefox	1	Web browser to generate HTTP traffic
Ethernet Interface (eth0)	1	Active network link for packet capture
Target (neverssl.com)	1	Remote HTTP web server for cleartext GET

3. Network Setup and Environment

Packet capture was executed on Kali Linux over interface eth0. Cleartext HTTP web traffic was generated by requesting http://neverssl.com (IP: 34.223.124.45) on TCP port 80.

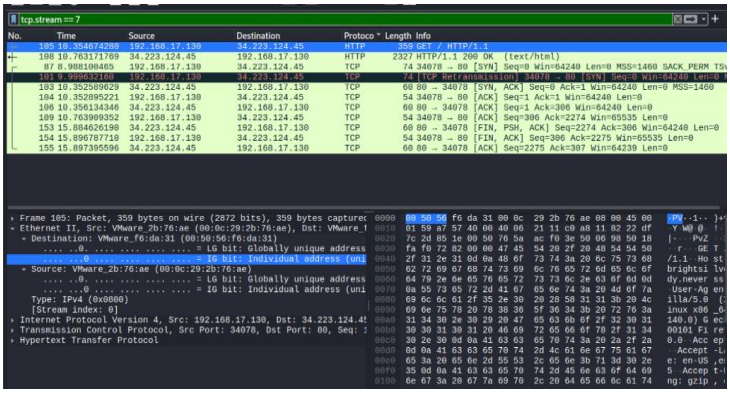


Figure 1: TCP Three-Way Handshake & HTTP GET Packet (tcp.stream eq 7)

4. Capture and Analysis Procedure

The following systematic procedure was executed to capture and inspect the network traffic:

1. Open Wireshark in Kali Linux with administrative privileges and select eth0 interface.
2. Start live packet capture to record incoming and outgoing frames.
3. Launch Mozilla Firefox and navigate to http://neverssl.com.
4. Wait for the webpage to fully load, then stop the packet capture in Wireshark.
5. Apply the display filter 'http' to locate the HTTP web traffic.
6. Inspect the HTTP GET request and examine the TCP stream (tcp.stream eq 7).

5. Transport Layer Analysis

The transport-layer protocol used is TCP. The client initiated communication from ephemeral port 34078, connecting to web server destination port 80. The three-way handshake established a reliable connection with synchronized sequence numbers prior to application-layer HTTP transfer.

Packet	Frame No.	Source (IP:Port)	Destination (IP:Port)	TCP Flags / Status
SYN	87	192.168.17.130:34078	34.223.124.45:80	[SYN] Seq=0
SYN-ACK	103	34.223.124.45:80	192.168.17.130:34078	[SYN, ACK] Ack=1
ACK	104	192.168.17.130:34078	34.223.124.45:80	[ACK] Seq=1, Ack=1
HTTP GET	105	192.168.17.130:34078	34.223.124.45:80	GET / HTTP/1.1
HTTP 200 OK	108	34.223.124.45:80	192.168.17.130:34078	200 OK (text/html)

6. HTTP Traffic Verification

The HTTP traffic verification was performed by applying the display filter 'http'. Wireshark isolated the application-layer request and server response:

Observed HTTP packets:

- Packet 105: GET / HTTP/1.1 (Client Request to neverssl.com)
- Packet 108: HTTP/1.1 200 OK (Server Response containing HTML body)

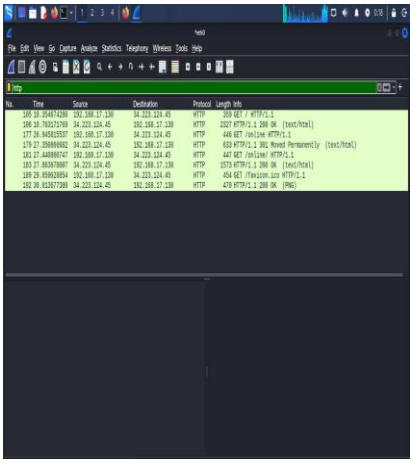


Figure 2: HTTP GET Request & HTTP 200 OK Response (http filter)

7. Observation Summary

Client / Source	Server / Destination	Protocol Event	Status
Client (Kali)	Server (34.223.124.45)	TCP 3-Way Handshake	ESTABLISHED
Client (Kali)	Server (34.223.124.45)	HTTP GET / HTTP 200 OK	SUCCESSFUL

8. Result

Network traffic was successfully captured and analyzed using Wireshark on Kali Linux. The TCP three-way handshake (SYN, SYN-ACK, ACK) and subsequent HTTP GET request and 200 OK response were identified successfully. All source and destination IP addresses and TCP port numbers were recorded from the capture.

9. Conclusion

This practical demonstrated how Wireshark can be used to inspect network protocols at the packet level. The analysis confirmed that TCP creates a reliable virtual circuit using sequence and acknowledgment numbers before application-layer HTTP traffic flows. Wireshark provides valuable visibility into OSI Layer 3 (Network) and Layer 4 (Transport) operations.

10. Submission Evidence Checklist

- ✓ Wireshark packet capture file: protocol_analysis.pcapng
- ✓ TCP Handshake Screenshot: screenshots/Figure_1_TCP_Handshake.png
- ✓ HTTP Traffic Screenshot: screenshots/Figure_2_HTTP_Traffic.png
- ✓ GitHub Repository: https://github.com/kawdeyash/Wireshark.git